

ViennaPhysicsEngine

Eight pages to your first simulated object.

ViennaPhysicsEngine computes rigid-body and cloth motion on the CPU. It does not open a window or draw anything. Start with a console program, then send its results to VVEngine. You use **import VEPysicsEngine;** and namespace **vpe**.

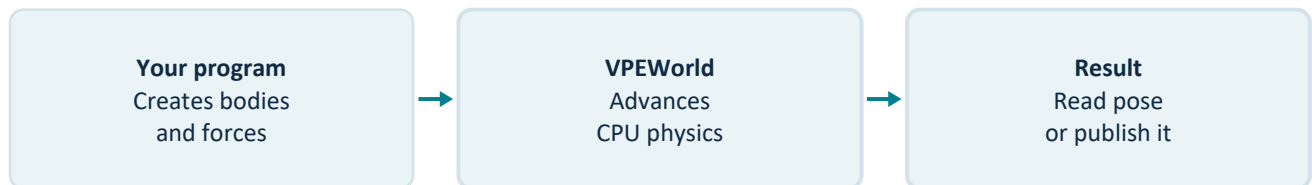
1. Install the tools

Install Git, Visual Studio with Desktop development with C++ and C++ CMake tools, and CMake 3.28 or newer. A recent MSVC with C++23 module support and Ninja is required. The standalone recipe uses bundled GLM and does not need Vulkan or VVEngine.

2. Clone the engine

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaPhysicsEngine.git
cd ViennaPhysicsEngine
```

Use the main branch, as with VVEngine. No branch switch or special clone option is needed. If the repository folder already exists, use that clone instead of cloning again.



Pages **2-3** build/link; **4** API; **5** complete falling-cube program; **6-7** callbacks, joints and cloth; **8** visual example and troubleshooting. The detailed physics manual explains the solver.

2. Compile the physics engine by itself

The first build does not need the visual example.

```
# From ViennaPhysicsEngine, in ordinary PowerShell:
.\build_cmake.cmd debug --without-vve --tests
```

The script initializes MSVC, configures Ninja, builds the module and runs the standalone physics checks. `--without-vve` turns off the renderer-dependent example. `--tests` turns tests on. Omit `--tests` for a library-only build; use `release` for optimized code.

Stage	What success means
Configure	CMake found compiler/dependencies and generated Ninja files.
Compile/link	The module implementation became a library.
Tests	The test executable ran and CTest reports its result.
Your application	Still needs its own executable target, source and linkage.

Direct CMake alternative

For these direct commands, open an **x64 Developer PowerShell for Visual Studio** so `cl` and `Ninja` are on `PATH`. Each trailing backtick continues the PowerShell command.

```
cmake -S . -B build/standalone -G Ninja `
  -DCMAKE_BUILD_TYPE=Debug `
  -DVPE_BUILD_EXAMPLES=OFF -DVPE_BUILD_TESTS=ON
cmake --build build/standalone
ctest --test-dir build/standalone --output-on-failure
```

What you link

`ViennaPhysicsEngine::ViennaPhysicsEngine` is the CMake target. `VEPhysicsEngine` is the module name in C++ source. They have different names and serve different jobs.

3. Link a console application

Keep your application beside the cloned library.

Create **C:/data/GitHub/MyPhysics**. Put main.cpp from page 5 and this CMakeLists.txt inside. The relative ../ViennaPhysicsEngine path assumes those folders are siblings.

```
cmake_minimum_required(VERSION 3.28)
project(MyPhysics LANGUAGES CXX)
set(VPE_BUILD_EXAMPLES OFF CACHE BOOL "" FORCE)
set(VPE_BUILD_TESTS OFF CACHE BOOL "" FORCE)
add_subdirectory(../ViennaPhysicsEngine
                 ${CMAKE_BINARY_DIR}/vpe)
add_executable(MyPhysics main.cpp)
target_link_libraries(MyPhysics PRIVATE
                     ViennaPhysicsEngine::ViennaPhysicsEngine)
```

In an x64 Developer PowerShell, run:

```
cd C:\data\GitHub\MyPhysics
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build
.\build\MyPhysics.exe
```

add_subdirectory builds the physics library in the same build as your program. target_link_libraries supplies the public module and C++23 settings. The target also supplies bundled GLM when no shared glm::glm target is present.

An import statement alone is insufficient: the module must be built and linked. Do not include VPE.ixx as a header, add the old VPE.hpp, or copy compiler module files between different compilers/builds.

Expected first result

The page 5 program prints a final cube height after about two seconds of simulation. No window appears because this standalone application has no renderer.

4. The essential physics API

Use a world, stable owners and body handles.

API / value	Meaning
vpe::VPEWorld	Owns the registered simulation objects; call tick(seconds).
VPEWorld::Body	Rigid object: pose, velocity, inverse mass and shape.
VPEWorld::g_cube	Built-in convex cube shape; pass &VPEWorld::g_cube.
addBody / eraseBody(shared_ptr)	Register/remove a rigid body; retain its shared_ptr.
Body::setForce / removeForce	Add/replace or remove a persistent force by ID.
Body::m_on_move / m_on_erase	Callbacks for visual updates and removal.
addConstraint	Register a ball socket, hinge, slider, fixed or distance joint.
Cloth / addCloth / clearCloths	Deformable mesh and its separate lifetime.
mass_inv = 0	Static object; positive inverse mass makes it dynamic.

Body borrows a world pointer, a Polytope pointer and a void* owner identity. Keep world, shape and owner alive. Give each normal body a unique stable owner address. shared_ptr keeps the Body alive, not those borrowed objects.

The built-in ground collision plane is $y=0$. Gravity must be installed explicitly on rigid bodies. Cloth has its own gravity path. Math aliases such as glmvec3 and glmquat are exported globally; the quaternion identity is {1,0,0,0} in w,x,y,z order.

Three beginner rules

Pass time in seconds. Use convex collision shapes. Remove a body through its stored shared_ptr; that overload also prunes its joints. Queue removal from collision callbacks until tick returns.

5. Example: a falling cube

Complete main.cpp for the project on page 3.

```
#include <iostream>
#include <memory>
import VEPHysicsEngine;

int main() {
    int owner = 1; // Stable identity.
    vpe::VPEWorld world;
    using W = vpe::VPEWorld;
    auto body = std::make_shared<W::Body>(
        &world, "cube", &owner, &W::g_cube,
        glmvec3{1,1,1}, glmvec3{0,4,0},
        glmquat{1,0,0,0}, glmvec3{0,0,0},
        glmvec3{0,0,0}, 1.0_real, 0.2_real, 0.8_real);
    body->setForce(0ul,
        W::Force{{0, world.c_gravity, 0}});
    world.addBody(body);
    for (int frame = 0; frame < 120; ++frame)
        world.tick(1.0/60.0);
    std::cout << "Final height: " << body->m_positionW.y
        << '\n';
    world.eraseBody(body); // Remove safely.
}
```

The body starts at height 4, uses cube geometry and inverse mass 1. Gravity is a persistent acceleration entry. tick advances fixed time slots internally; 120 calls represent about two seconds of incoming time. Watch m_positionW and m_linear_velocityW in the debugger. The cube falls toward the Y=0 ground and reacts to contact. The owner is a stack integer whose address remains stable. The world and built-in cube shape remain alive during the simulation.

A real game loop

Call world.tick(frameElapsedSeconds) from your game update. Do not also move the same dynamic object independently in rendering. Pause by skipping tick; avoid forwarding all paused time on resume.

6. Callbacks and joints

Small extensions to the falling-cube program.

Read or publish a body pose

Before addBody in page 5, install this callback. In a visible game, replace printing with your renderer's transform update.

```
body->m_on_move = [](double residual,
                    std::shared_ptr<W::Body> b) {
    std::cout << "y=" << b->m_positionW.y << '\n';
    // residual is time since the last fixed slot,
    // not the entire render-frame delta.
};
```

Connect two existing bodies

This recipe assumes bodyA and bodyB are registered shared_ptr<W::Body> values. The anchor is a world position; the axis must be nonzero.

```
auto hinge = std::make_shared<W::HingeJoint>(
    bodyA, bodyB, glmvec3{0,2,0}, glmvec3{0,1,0});
hinge->enableLimit(-0.5_real, 0.5_real); // Radians.
world.addConstraint(hinge);
// Later, detach the joint:
world.removeConstraint(hinge);
```

Joint	Use it when
BallSocketJoint	Bodies share a point and may rotate freely.
HingeJoint	Rotation should remain around one axis.
SliderJoint	Translation should remain along one axis.
FixedJoint	Relative pose should remain fixed.
DistanceConstraint	Body centers should keep a chosen separation.

Callbacks do not upload GPU buffers. Your game links a Body owner with its visible handle, copies the pose and lets VVEngine render. Do not delete owners/shapes or alter physics containers during callback traversal.

7. Example recipe: a pinned cloth

Add this after constructing world; include <vector>.

Use this in the page 5 project, before its simulation loop. The top two points are pinned. Triangle indices describe the mesh; the move callback returns its current original-order vertex positions.

```
int clothOwner = 2; // Stable identity.
std::vector<glmvec3> vertices{
    {-1,3,0}, {1,3,0}, {-1,1,0}, {1,1,0}};
std::vector<uint32_t> triangles{0,2,1, 1,2,3};
std::vector<glmvec3> pinned{vertices[0], vertices[1]};
auto onMove = [](double, std::shared_ptr<W::Cloth> cloth) {
    auto positions = cloth->generateVertices();
    // Copy positions to a deformable visual mesh here.
};
auto onErase = [](std::shared_ptr<W::Cloth>) {};
auto cloth = std::make_shared<W::Cloth>(
    &world, "sheet", &clothOwner, onMove, onErase,
    vertices, triangles, pinned, 0.1_real, 8, 0.8_real);
world.addCloth(cloth);
```

After the simulation loop, call **world.clearCloths()**; before destroying the owner or visual resources. Cloth needs finite vertices and valid, complete triangles. Pinned positions should match input positions. More substeps cost more CPU work; higher compliance permits more deformation.

generateVertices preserves the render mesh's original vertex order, including duplicate render vertices represented by one physical point. Your renderer must handle position-buffer and normal updates. clear for rigid bodies does not replace clearCloths.

First visible integration

Load a cloth mesh, create the Cloth from its positions/indices, update that mesh from the callback and keep both identities alive until removal. See the full physics manual before adding dense cloth and moving-body contact.

8. Visual demo and troubleshooting

Build the standalone example first; add rendering afterward.

For the visual demo, clone ViennaVulkanEngine next to ViennaPhysicsEngine and follow its quickstart. Install the Vulkan SDK and vcpkg, then run these from the respective repository roots:

```
# In ViennaVulkanEngine:
.\build_windows.cmd debug --no-tests
# In ViennaPhysicsEngine:
.\build_cmake.cmd debug
```

The example source used for this guide imports the VEngine renderer module. The current sibling uses VVEngine, so the visual bridge may need adaptation before this combined build works. Start with `--without-vve` for an isolated learning path. The script's expected visual output is `ViennaVulkanEngine/bin/debug/exe/physicsexample.exe`.

Symptom	Check
No movement	Positive inverse mass, gravity force and advancing tick.
No picture	Standalone physics has no rendering; attach a visible object/callback.
Crash after removal	Owner/shape lifetime; use <code>eraseBody(shared_ptr)</code> and queued deletion.
Very slow after pausing	Large incoming delta creates multiple catch-up steps.
Cloth creation fails	Finite geometry, triangle indices and positive substeps.

ID	Source file and starting line
R	README.md:25
C	CMakeLists.txt:161
M	modules/VPE.ixx:98
B	modules/VPE.ixx:461
F	modules/VPE.ixx:268
K	modules/VPE.ixx:1234
J	modules/VPE.ixx:2082
L	modules/VPE.ixx:3456

Guide updated: 4 October 2026. Setup uses the main branch. The falling-cube application was compiled, linked and run with MSVC/Ninja against the module API; final height was 0.5005. The visual bridge was not run. See the full manual for solver internals.